Stage 6 实验报告

于越洋

1 实验内容

我在 `backend/reg` 目录下添加了新的文件 `icrregalloc.py`，其中实现了图着色寄存器分配算法，并在 `backend/asm.py` 中调用新实现的寄存器分配算法。
我还构造了多个测例，用于对比图着色分配算法和原来的启发式分配算法的效果。

2 代码运行逻辑

2.1 初始化

2.1.1 scratch_regs

将 `t0~t3` 设定为 `self.scratch_regs`，它们不参与着色分配，而是用于spill 的临时装载和并行搬运寄存器时破坏。

2.1.2 colors 与 K

先定义保留寄存器集合 `self.reserved_idx`，包含 `SP/RA/ZERO/GP/TP` 和 `self.scratch_regs`。
`self.colors` 定义为 `emitter.allocatableRegs` 中去掉保留寄存器的集合。
图着色算法的颜色数为 `self.K = len(self.colors)`。

2.1.3 caller/callee 分类

定义 `self._callee_colors`，`self._caller_colors`，`self._other_colors` 用于后续选色的偏好策略：

- 跨调用的临时变量更偏好 callee-saved，从而减少 call 前保存/恢复
- 不跨调用的临时变量更偏好 caller-saved，从而避免无谓的被调者保存

2.1.4 其他

将冲突图、移动指令集合、节点集合等均初始化为空集合。

2.2 主入口及分配流程

`self.accept(graph, info)` 为寄存器分配算法的入口，它包含以下流程：

1. `_collect_temps_and_costs(graph)`，收集临时变量和 spill 代价；

2. `_init_state()`，初始化图结构和每个节点的状态；
3. `_build(graph)`，根据活跃信息构建冲突图，并收集 move 指令用于后续合并；
4. `_mk_worklist()`，把节点分入 simplify / freeze / spill 三类工作表；
5. 主循环：依次进行 simplify，coalesce，freeze，selectSpill；
6. `_assign_colors()`：进行图着色（寄存器分配）；
7. `_mark_used_after_coloring()`：标记用到的物理寄存器，便于函数序言/尾声保存 callee-saved
8. 遍历基本块，调用 `_emit_basic_block()` 生成最终汇编代码。

2.3 收集临时变量和 spill 代价

首先调用 `add_temp_obj()` 把出现在 srcs/dsts 里的临时变量加入 `self.temps`，作为图节点。
对于基本块权重，由于 `BasicBlock` 中未实现对循环深度的记录，因此使用默认值 `1`。
最后统计跨调用活跃的临时变量，以便在选色时设定偏好，即将跨调用的临时变量尽量放入 callee-saved 寄存器。

2.4 初始化图结构和节点状态

把冲突图、move 相关的集合、所有工作表、节点集合均清空，节点状态（邻接表、度数等）也均清空。

2.5 构建冲突图

将活跃集合设定为该基本块末尾活跃变量集合，即 `live = bb.liveOut`；
逆序遍历 loc，设置 `defs = instr.getWritten()` 和 `uses = instr.getRead()`；
若 `instr` 是 `Riscv.Move`，则先从 `live` 中去除 `uses` 中的活跃变量，再将该 move 指令加入 `worklist_moves` 和 `move_list[n]`；
对每个 def 的变量和其后仍然活跃的所有变量建立冲突边；
最后更新 `live` 集合，从中去除 `defs`，并添加 `uses`。

2.6 分配工作表

对每个节点，做以下判断：

- 若度数大于 `K`，则分入 `spill_worklist`
- 若度数不大于 `K`，但还与 move 指令相关，则分入 `freeze_worklist`
- 否则分入 `simplify_worklist`

2.7 主循环

2.7.1 Simplify

从 `simplify_worklist` 中弹出一个节点，将其加入 `select_stack`，并对所有相邻节点进行度数减 1，相当于将弹出的节点从图中删除。

2.7.2 Coalesce

从 `worklist_moves` 中取出一条 move 指令，找到其 `src` 和 `dst` 的代表元 `v` 和 `u`，做以下判断：

- 如果 `u==v`：move 已经等价，放入 `coalesced_moves`，并调用 `_add_worklist(u)` 将 `u` 加入 `simplify_worklist`；
- 如果 `(u,v)` 是冲突图中的一条边：两者冲突，move 放入 `constrained_moves`，并尝试把 u/v 重新加入工作表；
- 否则：用 `_conservative(adjacent(u) | adjacent(v))` 判断是否“保守合并”，若是则 `_combine(u,v)`，否则 move 进入 `active_moves`。

2.7.3 Conservative

统计传入的节点集合中度数大于等于 `K` 的节点数量，若该数量小于 `K`，则判定为“保守合并”。

2.7.4 Combine

合并节点 `v` 和 `u`，主要完成以下操作：

- 把 `v` 从 `freeze_worklist` 和 `spill_worklist` 中移出，加入 `coalesced_nodes`；
- 设定 `alias[v]=u`；
- 合并两个节点的 moveList，将 `v` 的 `move_list` 中的元素加入 `u` 的 `move_list`；
- 对 `v` 的相邻点 `t`，在冲突图中添加从 `u` 到 `t` 的边，并为 `t` 的度数减 1；
- 如果 `u` 的度数因此变成高度数，则把 `u` 推入 `spill_worklist`；
- 如果 v 跨调用活跃，则 u 也标记跨调用活跃。

2.7.5 Freeze

从 `freeze_worklist` 取节点 `u`，放入 `simplify_worklist`，并将 `u` 的相关 move 全部冻结，必要时将另一端节点 `v` 移入 `simplify_worklist`。

2.7.6 SelectSpill

在 `spill_worklist` 中选 `score = spill_cost / degree` 最小者，把它移到 `simplify_worklist`，并冻结它相关的 move。

2.7.8 RewriteProgram

为了方便实现，我将 spill 的 load/store 插入延后到汇编发射阶段，因此不进行 RewriteProgram 后的再次构图迭代，其效果等价于在使用/定义处插入访存。

2.8 进行图着色

循环执行以下过程：

- 从 `select_stack` 中选取节点 `n`，将其出栈，设定可用颜色 `ok_colors` 为所有颜色；
- 逐个检查 `n` 的邻居，若邻居已经着色，则从 `ok_colors` 中删除邻居的颜色；
- 若 `ok_colors` 为空，则将 `n` 加入 `spilled_nodes`；
- 否则调用 `_choose_color_with_bias()` 选择一个可用的颜色分配给 `n`；
  - 这里选择颜色时，如果节点 `n` 有相关的 move，会优先选择 move 对端的颜色，从而便于后续将 move 消掉；如果没有相关的 move，则若 `n` 跨调用活跃，则会更倾向于分配 callee-saved 颜色。
- 对已合并的节点，继承其代表元的颜色；若其代表元在 `spilled_nodes` 中，则该合并节点也加入 `spilled_nodes`。

2.9 标记已用寄存器

遍历 `self.color_of.values()`，对出现过的物理寄存器设置 `used=True`（便于函数调用时 callee-saved 寄存器的保存）。此外，scratch_regs 也标记为 `used=True`。

2.10 生成汇编代码

2.10.1 基本块内的 spill_cache

`_emit_basic_block()` 为每个基本块维护：

- `spill_cache`：当前块内某个 spilled temp 被装载到哪个 scratch；
- `spill_dirty`：该 spilled temp 的 scratch 内容被写过，需要在合适时机 store 回栈。

处理策略：

- 读 spilled temp：若不在 cache 中，就分配一个 scratch 并 load；若在 cache 中就直接复用；
- 写 spilled temp：分配/复用 scratch，但标记 dirty（延迟写入）；
- 在基本块结束或遇到调用指令时清除 dirty，即完成延迟写。

2.10.2 函数参数的处理

`pending_params` 收集连续的 `Riscv.Parameter`，直到遇到非 Parameter 才一次性 `_emit_parameters_batch()`：

- 对寄存器参数：若该 temp 被 spill，则直接 store 入栈槽；否则直接 move 到目标寄存器；
- 对栈参数：设置 offsets，并在需要时 load 到目标寄存器。

2.10.3 函数调用的处理

对于函数调用，进行以下处理：

- 保存跨调用活跃的 caller-saved 寄存器
- 准备参数
  - 对于通过寄存器传递的参数，存入相应的参数寄存器
  - 对于通过栈传递的参数，压入栈中传递，并相应地调整栈指针
- 实现 `call` 指令
- 将返回值写入目标寄存器
- 回收传参使用的栈空间，并恢复之前保存的 caller-saved 寄存器

2.10.4 普通指令的处理

对于普通指令，进行以下处理：

- 如果是 move 指令，且两端的代表元或着色结果相同，则直接跳过
- 对每个 src：
  - 若已经是寄存器，则直接使用
  - 若是临时变量，则判断是否 spilled，若 spilled，则从 spill_cache 中申请一个暂存寄存器，并加载临时变量到该暂存寄存器；否则直接使用其代表元的着色寄存器。
- 对每个 dst，与 src 处理类似，只是申请暂存寄存器后不需要加载临时变量到该寄存器。
- 最后调用 `instr.fillRegs(dst_regs, src_regs)` 。

3 举例说明

我构造了如下几个测例来体现图着色寄存器分配算法的效果

3.1 双端复制

使用的测例为

```
int double_copy(int a, int b, int c, int d) {
    int x = a + b;
    int y = x;
    int z = c + d;
    int w = z;
    return y + w;
}
```

旧算法生成的汇编代码为

```
double_copy:
    # start of prologue
    addi sp, sp, -48
```

```
    sw ra, 44(sp)
    # end of prologue

    # start of body
    add t0, a0, a1
    mv t1, t0
    mv t0, t1
    add t1, a2, a3
    mv t2, t1
    mv t1, t2
    add t2, t0, t1
    mv a0, t2
    j double_copy_exit
    # end of body
```

新代码生成的汇编代码为

```
double_copy:
    # start of prologue
    addi sp, sp, -48
    sw ra, 44(sp)
    # end of prologue

    # start of body
    mv t5, a1
    mv t6, a2
    mv t4, a3
    add t5, a0, t5
    add t4, t6, t4
    add t4, t5, t4
    mv a0, t4
    j double_copy_exit
    # end of body
```

可见在旧算法中，赋值语句本身会生成一条 `mv` 指令，而后为其他语句“腾出”寄存器时又会生成 `mv` 指令，存在不必要的复制；而新算法中消除了这种不必要的 `mv` 语句。

3.2 分支合流

使用的测例为

```
int branch_copy(int a, int b, int cond) {
    int x;
    if (cond) {
        int t = a + 1;
```

```
        x = t;
    } else {
        int t = b + 2;
        x = t;
    }
    return x + 3;
}
```

旧算法生成的汇编代码为

```
branch_copy:
    # start of prologue
    addi sp, sp, -60
    sw ra, 44(sp)
    # end of prologue

    # start of body
    sw a0, 48(sp)
    sw a1, 52(sp)
    beq x0, a2, _L1
    li t0, 1
    lw t1, 48(sp)
    add t2, t1, t0
    mv t0, t2
    mv t1, t0
    sw t1, 56(sp)
    j _L2
_L1:
    li t0, 2
    lw t1, 52(sp)
    add t2, t1, t0
    mv t0, t2
    mv t1, t0
    sw t1, 56(sp)
_L2:
    li t0, 3
    lw t1, 56(sp)
    add t2, t1, t0
    mv a0, t2
    j branch_copy_exit
    # end of body
```

新算法生成的汇编代码为

```
branch_copy:
    # start of prologue
    addi sp, sp, -48
    sw ra, 44(sp)
    # end of prologue

    # start of body
    mv t6, a0
    mv t5, a1
    mv t4, a2
    beq x0, t4, _L1
    li t4, 1
    add t4, t6, t4
    j _L2
_L1:
    li t4, 2
    add t4, t5, t4
_L2:
    li t5, 3
    add t4, t4, t5
    mv a0, t4
    j branch_copy_exit
    # end of body
```

可以看到，旧算法是基于基本块局部的分配，在基本块边界上固定对所有活跃变量进行压栈，并解除其与寄存器的绑定，导致出现了多次不必要的 `sw` 和 `lw`；而新算法在整个函数范围内进行冲突分析和着色，使得局部变量能够始终保持在寄存器中，减少访存次数。

3.3 跨调用活跃

使用的测例为

```
int live_across_call(int a, int b, int c) {
    int x = a + b;
    int y = x + c;
    int z = g(y);
    return z + x;
}
```

旧算法生成的汇编代码为

```
live_across_call:
    # start of prologue
    addi sp, sp, -68
```

```
    sw ra, 44(sp)
    # end of prologue

    # start of body
    add t0, a0, a1
    mv t1, t0
    add t0, t1, a2
    mv t2, t0
    sw a0, 48(sp)
    mv a0, t2
    sw t0, 52(sp)
    sw t1, 56(sp)
    sw a1, 60(sp)
    sw a2, 64(sp)
    call g
    mv t0, a0
    mv t1, t0
    lw t0, 56(sp)
    add t2, t1, t0
    mv a0, t2
    j live_across_call_exit
    # end of body
```

新算法生成的汇编代码为

```
live_across_call:
    # start of prologue
    addi sp, sp, -48
    sw ra, 44(sp)
    sw s1, 0(sp)
    # end of prologue

    # start of body
    mv t6, a0
    mv t5, a1
    mv t4, a2
    add s1, t6, t5
    add t4, s1, t4
    mv a0, t4
    call g
    mv t4, a0
    add t4, t4, s1
    mv a0, t4
    j live_across_call_exit
    # end of body
```

在这个测例中，`x` 在调用函数 `g()` 后仍需要被使用，即它是一个跨调用活跃变量。
旧算法生成的代码将 `x` 放入 caller-saved 寄存器 `t1`，这导致在调用其他函数时，它需要被保存在栈中，调用后需要恢复，且 `a1`、`a2` 等本不需要再被使用的寄存器也被保存，造成了多次不必要的内存访问。
新算法直接将 `x` 放入 callee-saved 寄存器 `s1`，从而只需要在函数入口处保存一次 `s1`，出口处恢复 `s1`，当连续调用多个函数时，新算法可以优化掉大量不必要的保存和恢复，大幅减少内存访问次数。

4 运行时间对比

采用 6 个测例（附在报告末尾）进行对比，分别使用旧的启发式寄存器分配算法和新的图着色寄存器分配算法编译这 6 个测例，将所得的汇编代码各运行 50 次，记录平均运行时间，得到如下表所示的结果

测例	特点	启发式代码运行时间 (ms)	图着色代码运行时间 (ms)	新旧运行时间之比
`many_live_variables`	含有大量活跃变量	33.062	14.077	0.426
`cold_variables`	部分变量在循环外部活跃（冷变量），部分在循环内部活跃（热变量）	21.121	10.539	0.499
`many_branches`	含有大量分支语句	22.947	14.497	0.632
`many_calls`	含有大量函数调用	46.781	30.521	0.652
`many_moves`	含有大量复制指令	134.309	43.776	0.326
`quicksort`	快速排序，用于综合测试	285.927	165.101	0.577

由表中结果可见，采用图着色寄存器分配算法后，代码运行时间相较于原来少了一半左右。

5 附：运行时间对比使用的测例

5.1 `many_live_variables`

```
int test_many_live_variables() {
    int a = 1;
    int b = 2;
    int c = 3;
    int d = 4;
    int e = 5;
    int f = 6;
    int g = 7;
```

```
    int h = 8;
    int i = 9;
    int j = 10;
    int k = 11;
    int l = 12;
    int m = 13;
    int n = 14;
    int o = 15;
    int p = 16;
    int q = 17;
    int r = 18;
    int s = 19;
    int t = 20;


    int result = 0;

    for (int x = 0; x < 1000000; x = x + 1) {
        result = result + a + b + c + d + e + f + g + h + i + j
                  + k + l + m + n + o + p + q + r + s + t;

        a = a + 1;
        b = b + 2;
        c = c + 3;
        d = d + 4;
        e = e + 5;
        f = f + 6;
        g = g + 7;
        h = h + 8;
        i = i + 9;
        j = j + 10;
        k = k + 11;
        l = l + 12;
        m = m + 13;
        n = n + 14;
        o = o + 15;
        p = p + 16;
        q = q + 17;
        r = r + 18;
        s = s + 19;
        t = t + 20;
    }

    return result;
}
```

```
int main() {
    return test_many_live_variables();
}
```

5.2 `cold_variables`

```
int main() {
    int OUT = 200;
    int IN  = 6000;

    int c0 = 1;
    int c1 = 2;
    int c2 = 3;
    int c3 = 4;
    int c4 = 5;
    int c5 = 6;
    int c6 = 7;
    int c7 = 8;

    int sum = 0;

    for (int o = 0; o < OUT; o = o + 1) {
        int base = c0 + c1 + c2 + c3 + c4 + c5 + c6 + c7;

        int a = base + 11;
        int b = base + 13;
        int d = base + 17;
        int e = base + 19;
        int f = base + 23;
        int g = base + 29;

        for (int i = 0; i < IN; i = i + 1) {
            int t0 = a + b;
            int t1 = d + e;
            int t2 = f + g;
            int t3 = t0 + t1;
            int t4 = t2 + t3;
            int t5 = t4 + a;
            int t6 = t5 + d;
            sum = sum + t6;

            a = a + 3;
            b = b + 5;
            d = d + 7;
            e = e + 11;
```

```
            f = f + 13;
            g = g + 17;

            if (sum > 1000000000) sum = sum - 1000000000;
        }

        c0 = c0 + base;
        c1 = c1 + (sum % 7);
        c2 = c2 + (sum % 11);
        c3 = c3 + (sum % 13);
        c4 = c4 + 1;
        c5 = c5 + 2;
        c6 = c6 + 3;
        c7 = c7 + 4;

        if (c0 > 1000000000) c0 = c0 - 1000000000;
    }

    return sum + c0 + c1 + c2 + c3 + c4 + c5 + c6 + c7;
}
```

5.3 `many_branches`

```
int test_many_branches() {
    int a = 0;
    int b = 1;
    int c = 2;
    int d = 3;
    int result = 0;

    for (int x = 0; x < 1000000; x = x + 1) {
        if (x % 2 == 0) {
            a = a + 1;
            if (x % 3 == 0) {
                b = b + 2;
            } else {
                c = c + 3;
            }
        } else {
            d = d + 4;
            if (x % 5 == 0) {
                a = a - 1;
            } else if (x % 7 == 0) {
                b = b - 2;
            } else {
```

```
                c = c - 3;
            }
        }

        result = result + a + b + c + d;

        if (result % 100 == 0) {
            d = d + 1;
        }
    }

    return result;
}

int main() {
    return test_many_branches();
}
```

5.4 `many_calls`

```
int callee12(int a0, int a1, int a2, int a3,
             int a4, int a5, int a6, int a7) {
    int x = a0 + a2 + a4 + a6;
    int y = a1 + a3 + a5 + a7;
    int r = x + y;
    if (r > 1000000000) r = r - 1000000000;
    return r;
}

int main() {
    int N = 1200000;

    int x0 = 1;
    int x1 = 2;
    int x2 = 3;
    int x3 = 4;
    int x4 = 5;
    int x5 = 6;

    int acc = 0;

    for (int i = 0; i < N; i = i + 1) {
        int p = x0 + x1 + i;
        int q = x2 + x3 + i;
        int r = x4 + x5 + i;
```

```
        int ret = callee12(p, q, r, x0,
                           x1, acc, i, p + r);

        acc = acc + ret + x0 + x1 + x2 + x3 + x4 + x5;

        x0 = x0 + 1;
        x1 = x1 + 2;
        x2 = x2 + 3;
        x3 = x3 + 4;
        x4 = x4 + 5;
        x5 = x5 + 6;

        if (acc > 1000000000) acc = acc - 1000000000;
    }

    return acc + x0 + x1;
}
```

5.5 `many_moves`

```
int many_moves() {
    int N = 8000000;

    int a = 1;
    int b = 2;
    int c = 3;
    int d = 4;
    int sum = 0;

    for (int i = 0; i < N; i = i + 1) {
        int x = a + b;
        int y = x;
        int z = y + c;
        int w = z;

        int m;
        if (i % 2) {
            int t = w + d;
            m = t;
        } else {
            int t = w + a;
            m = t;
        }
```

```
        int u = m;
        sum = sum + u;

        a = a + 1;
        b = b + 2;
        c = c + 3;
        d = d + 4;

        if (sum > 1000000000) sum = sum - 1000000000;
    }

    return sum + a + b + c + d;
}

int main() {
    return many_moves();
}
```

5.6 `quicksort`

```
int n = 1000000;
int arr[1000000];

int qsort(int a[1000000], int l, int r) {
    int i = l;
    int j = r;
    int p = a[(l+r)/2];
    int flag = 1;
    while (i <= j) {
        while (a[i] < p) i = i + 1;
        while (a[j] > p) j = j - 1;
        if (i > j) break;
        int u = a[i]; a[i] = a[j]; a[j] = u;
        i = i + 1;
        j = j - 1;
    }
    if (i < r) qsort(a, i, r);
    if (j > l) qsort(a, l, j);
    return 0;
}

int state;
int rand() {
    state = (state * 64013 + 1531011) % 32768;
    return state % 1000;
```

```
}

int initArr(int n) {
    int i = 0;
    while (i < n) {
        arr[i] = rand();
        i = i + 1;
    }
    return 0;
}

int isSorted(int n) {
    int i = 0;
    while (i < n-1) {
        if ( arr[i] > arr[i+1] )
            return 0;
        i = i + 1;
    }
    return 1;
}

int main() {
    initArr(n);
    int sorted_before = isSorted(n);
    qsort(arr, 0, n-1);
    int sorted_after = isSorted(n);
    if (!(sorted_before==0 && sorted_after==1))
        return 1;
    return 0;
}
```